Rapport de Projet

Comparaison Pipeline CI/CD
Jenkins vs AWS CodePipeline

Application : Spring PetClinic (Java 21 / Spring Boot 4)

Auteur : **Aymen Achagui**

Date : 10 février 2026

Table des matières

1 Introduction **2**
1.1 Contexte . . . 2
1.2 Objectifs . . . 2
1.3 Application cible — Spring PetClinic . . . 2

2 Architecture **3**
2.1 Vue d'ensemble . . . 3
2.2 Stack technique . . . 4

3 Pipeline Jenkins **5**
3.1 Infrastructure . . . 5
3.2 Stages du pipeline . . . 5
3.3 Résultats d'exécution . . . 6

4 CodePipeline **7**
4.1 Infrastructure . . . 7
4.2 Stages du pipeline . . . 7
4.3 Auto-trigger sur push GitHub . . . 8
4.4 Résultats CodeBuild . . . 8
4.5 Vue du pipeline CodePipeline . . . 9

5 Analyse Qualité **10**
5.1 Configuration . . . 10
5.2 Résultats . . . 11

6 Registry Docker — AWS ECR **12**
6.1 Configuration . . . 12
6.2 Images publiées . . . 12

7 Comparaison Jenkins vs AWS CodePipeline **13**
7.1 Analyse . . . 13

8 Résultats & Conclusion **14**
8.1 Résultats obtenus . . . 14
8.2 Difficultés rencontrées . . . 14
8.3 Conclusion . . . 14

1. Introduction

1.1 Contexte

Ce projet a été réalisé dans le cadre d'une démarche d'apprentissage pratique des outils DevOps modernes. L'objectif était de ne pas se contenter de la théorie, mais de mettre en place de bout en bout deux pipelines CI/CD distincts sur une même application Java, afin de comprendre concrètement les implications de chaque approche en termes de configuration, de maintenance et d'intégration avec les services cloud.

L'application choisie est **Spring PetClinic**, une application de référence de l'écosystème Spring, suffisamment représentative d'un projet réel pour que les conclusions soient transposables en contexte professionnel.

1.2 Objectifs

Ce projet a pour objectifs de :

- Mettre en place deux pipelines CI/CD distincts pour une même application Java.
- Comparer les approches **self-hosted** (Jenkins) et **fully managed** (AWS CodePipeline).
- Automatiser la construction, les tests, l'analyse qualité et la publication d'images Docker.
- Évaluer les avantages et inconvénients de chaque solution en contexte réel.

1.3 Application cible — Spring PetClinic

L'application utilisée est **Spring PetClinic**, une application de référence de l'écosystème Spring.

- Langage : Java 21
- Framework : Spring Boot 4
- Build : Maven
- Tests : 57 tests unitaires JUnit

2. Architecture

2.1 Vue d'ensemble

Les deux pipelines partagent le même dépôt source (`GitHub`) et la même registry de destination (**AWS ECR**), mais diffèrent dans les outils d'orchestration et d'exécution. Ce choix est délibéré : il garantit une comparaison équitable où la seule variable est le pipeline lui-même, pas l'application ni la destination finale.

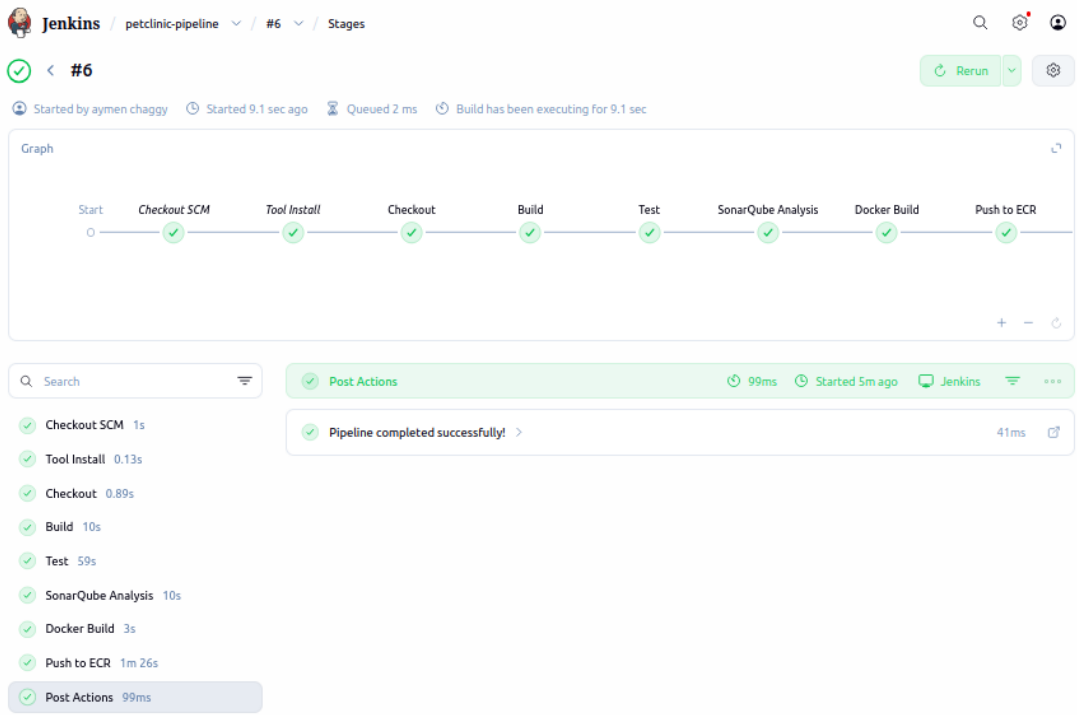

FIGURE 2.1 – Vue d'ensemble : deux pipelines, une même destination (AWS ECR)

2.2 Stack technique

Composant	**Technologie**
Application	Java 21 / Spring Boot 4
Build	Maven
Conteneurisation	Docker
Analyse qualité	SonarQube
Registry	AWS ECR
Pipeline (opt. 1)	Jenkins
Pipeline (opt. 2)	AWS CodePipeline + CodeBuild
Source	GitHub

3. Pipeline Jenkins

3.1 Infrastructure

Le pipeline Jenkins repose sur une infrastructure **self-hosted** orchestrée via **Docker Compose**, qui déploie deux services interconnectés sur un réseau Docker interne :

— **Jenkins** : serveur CI/CD principal, accessible sur le port 8080.
— **SonarQube** : plateforme d'analyse de la qualité du code, accessible sur le port 9000.

Les deux services communiquent via le réseau interne Docker. Jenkins désigne SonarQube par son nom de service (`http://sonarqube:9000`) et non par `localhost`, ce qui est un point de configuration important à ne pas confondre.

Le socket Docker de la machine hôte (`/var/run/docker.sock`) est monté dans le conteneur Jenkins, permettant au pipeline de construire et pousser des images Docker sans nécessiter un daemon Docker supplémentaire (pattern Docker-in-Docker).

3.2 Stages du pipeline

Le pipeline Jenkins est composé des stages suivants :

1. **Checkout SCM** — Récupération automatique du dépôt depuis Jenkins.
2. **Tool Install** — Installation des outils nécessaires (Maven, JDK).
3. **Checkout** — Clonage explicite du dépôt GitHub.
4. **Build** — Compilation du projet avec Maven (`mvn clean package -DskipTests`).
5. **Test** — Exécution des tests unitaires JUnit.
6. **SonarQube Analysis** — Analyse statique du code via SonarQube Scanner.
7. **Docker Build** — Construction de l'image Docker de l'application.
8. **Push to ECR** — Publication de l'image dans AWS Elastic Container Registry.

3.3 Résultats d'exécution

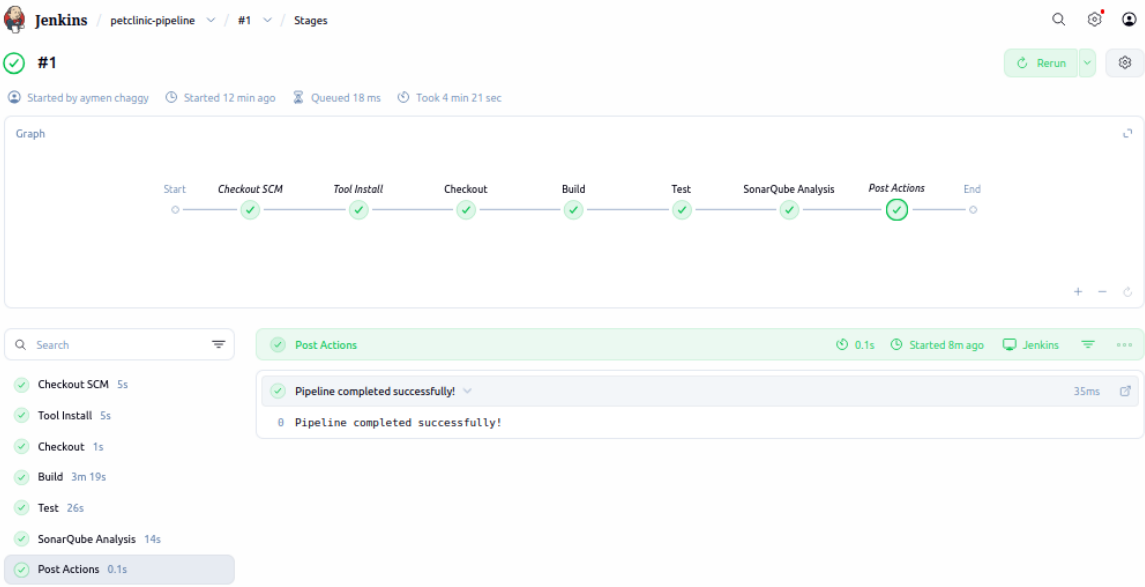

FIGURE 3.1 – Jenkins — Pipeline #6 complété avec succès (4 min 21 sec)

Le build #6 a été exécuté avec succès en **4 minutes 21 secondes**, avec tous les stages au vert.

Stage	Durée
Checkout SCM	1 s
Tool Install	0.13 s
Checkout	0.89 s
Build	10 s
Test	59 s
SonarQube Analysis	10 s
Docker Build	3 s
Push to ECR	1 min 26 s
Post Actions	99 ms

4. CodePipeline

4.1 Infrastructure

Le pipeline AWS est entièrement **managé** et repose sur deux services AWS :

— **AWS CodePipeline** : orchestrateur du pipeline.
— **AWS CodeBuild** : environnement d'exécution du build.

La configuration du build est définie dans un fichier `buildspec.yml` versionné dans le dépôt. Ce fichier déclare les phases `pre_build`, `build` et `post_build`, correspondant respectivement à l'authentification ECR, la compilation Maven, et la construction puis le push de l'image Docker.

4.2 Stages du pipeline

1. **Source** — Connexion à GitHub via GitHub App. Déclenché automatiquement à chaque push sur `main`.
2. **Build (CodeBuild)** — Compilation Maven, tests, construction et push de l'image Docker vers ECR.

4.3 Auto-trigger sur push GitHub

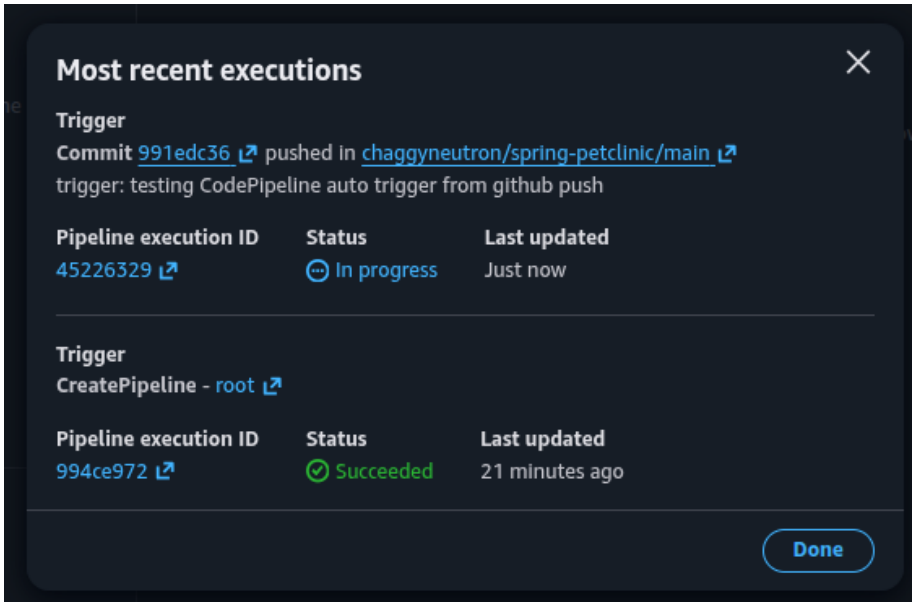

FIGURE 4.1 – Déclenchement automatique du pipeline sur push GitHub

La connexion entre GitHub et CodePipeline est établie via la **GitHub App** fournie par AWS. Contrairement à un webhook classique, cette intégration ne nécessite aucune configuration manuelle côté GitHub : AWS gère l'abonnement aux événements push directement depuis la console CodePipeline.

4.4 Résultats CodeBuild

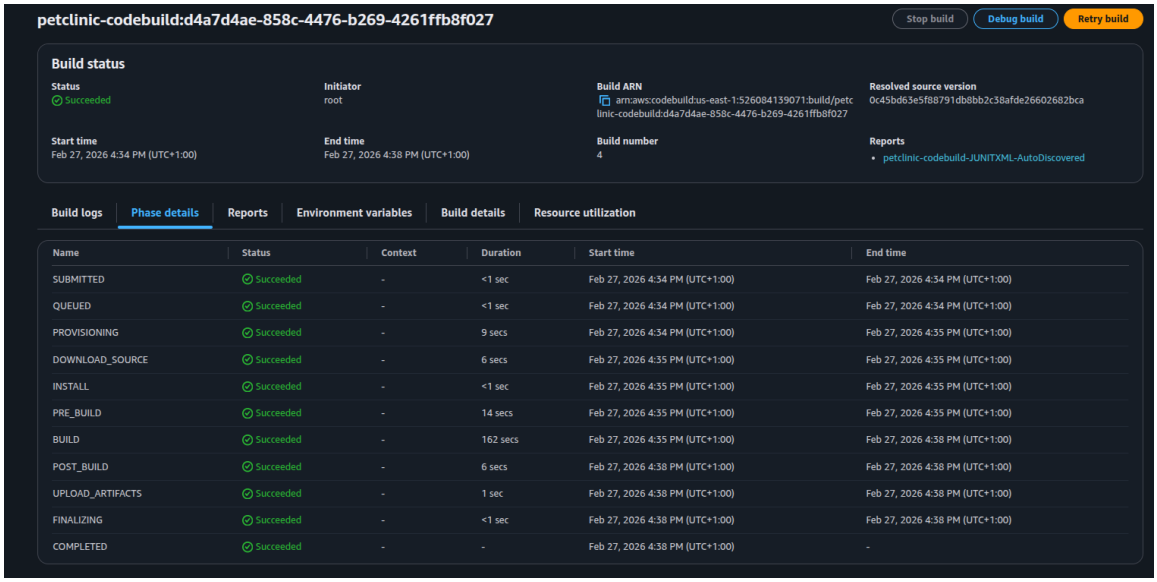

FIGURE 4.2 – AWS CodeBuild — Build #4 réussi (phases détaillées)

Phase	Statut	Durée
SUBMITTED	Succeeded	<1 s
QUEUED	Succeeded	<1 s
PROVISIONING	Succeeded	9 s
DOWNLOAD_SOURCE	Succeeded	6 s
INSTALL	Succeeded	<1 s
PRE_BUILD	Succeeded	14 s
BUILD	Succeeded	162 s
POST_BUILD	Succeeded	6 s
UPLOAD_ARTIFACTS	Succeeded	1 s
FINALIZING	Succeeded	<1 s
COMPLETED	Succeeded	—

4.5 Vue du pipeline CodePipeline

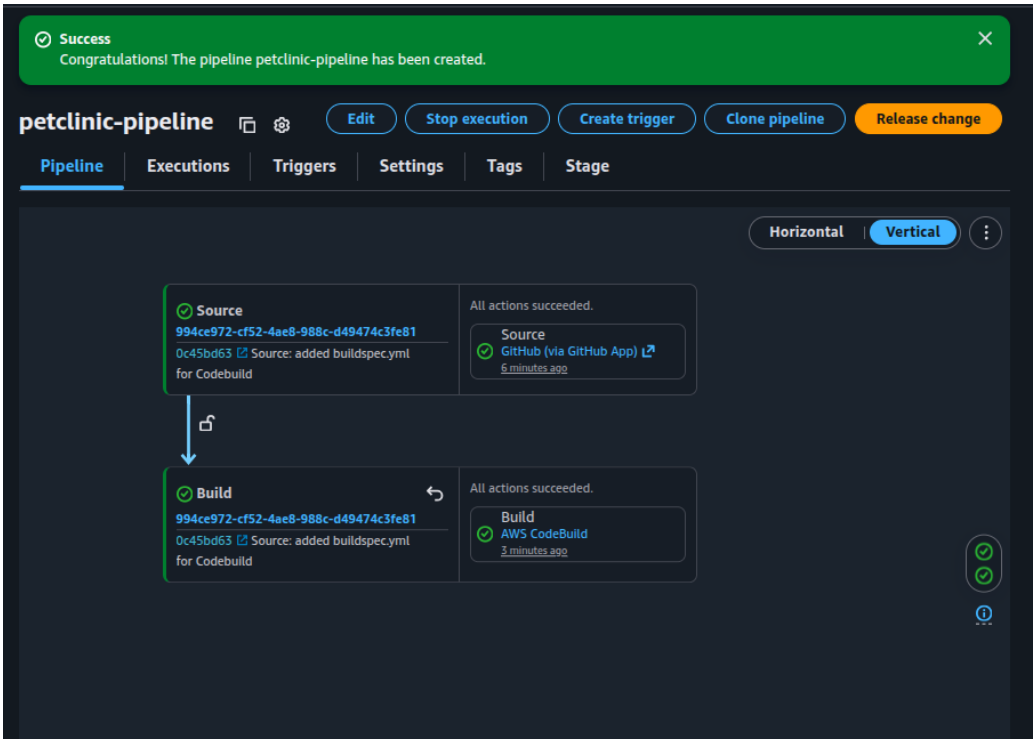

FIGURE 4.3 – AWS CodePipeline — petclinic-pipeline (Source + Build)

5. Analyse Qualité

5.1 Configuration

SonarQube a été intégré dans le pipeline Jenkins via le **plugin SonarQube Scanner**. La configuration s'est faite en trois étapes :

1. **Réseau Docker** : Jenkins et SonarQube étant deux conteneurs distincts dans le même `docker-compose.yml`, ils partagent un réseau interne commun. L'URL configurée dans Jenkins pointe sur le nom du service Docker (`http://sonarqube:9000`) et non sur `localhost`.
2. **Token d'authentification** : un token utilisateur a été généré depuis l'interface SonarQube (*My Account* > *Security*) et stocké comme credential de type *Secret text* dans Jenkins. Le Jenkinsfile référence ce credential par son ID sans exposer la valeur en clair dans le code source.
3. **Déclaration dans Jenkins** : le serveur SonarQube a été déclaré dans *Manage Jenkins* > *Configure System* > *SonarQube servers*, ce qui permet au plugin d'injecter automatiquement les variables d'environnement nécessaires (`SONAR_HOST_URL`, `SONAR_AUTH_TOKEN`) dans le stage d'analyse.

5.2 Résultats

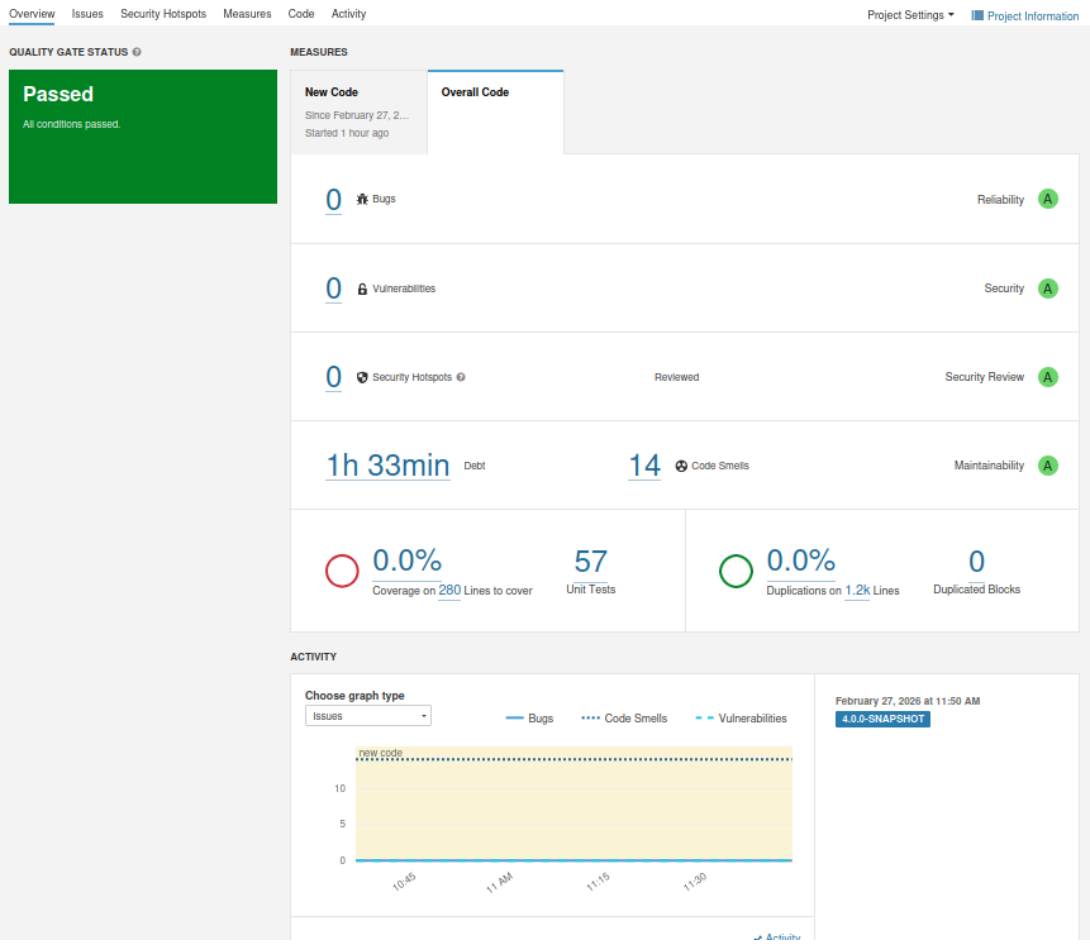

FIGURE 5.1 – SonarQube — Tableau de bord qualité du projet

Le Quality Gate est **Passed** avec les métriques suivantes :

Métrique	Valeur	Note
Bugs	0	A
Vulnérabilités	0	A
Security Hotspots	0 (Reviewed)	A
Code Smells	14	A
Dette technique	1h 33 min	A
Tests unitaires	57	—
Couverture	0.0%	—
Duplications	0.0%	—

Note : La couverture de code est à 0% car JaCoCo n'a pas été configuré dans cette phase. Les 57 tests unitaires s'exécutent et passent, mais leur couverture n'est pas instrumentée. C'est la prochaine amélioration prévue.

6. Registry Docker — AWS ECR

6.1 Configuration

Le repository ECR `petclinic` a été créé depuis la console AWS dans la région `us-east-1`. Deux configurations de permissions ont été nécessaires :

— **Pour Jenkins** : les credentials AWS (`AWS_ACCESS_KEY_ID` et `AWS_SECRET_ACCESS_KEY`) d'un utilisateur IAM avec les droits `ecr:GetAuthorizationToken`, `ecr:BatchCheckLayerAvailab` `ecr:PutImage` et `ecr:InitiateLayerUpload` ont été stockés dans Jenkins et injectés dans le pipeline via la directive `withCredentials`.

— **Pour CodeBuild** : un **IAM Role** dédié a été attaché au projet CodeBuild. Ce rôle inclut la politique `AmazonEC2ContainerRegistryPowerUser`, ce qui permet à CodeBuild de s'authentifier et de pousser des images sans manipuler de credentials explicites dans le code.

6.2 Images publiées

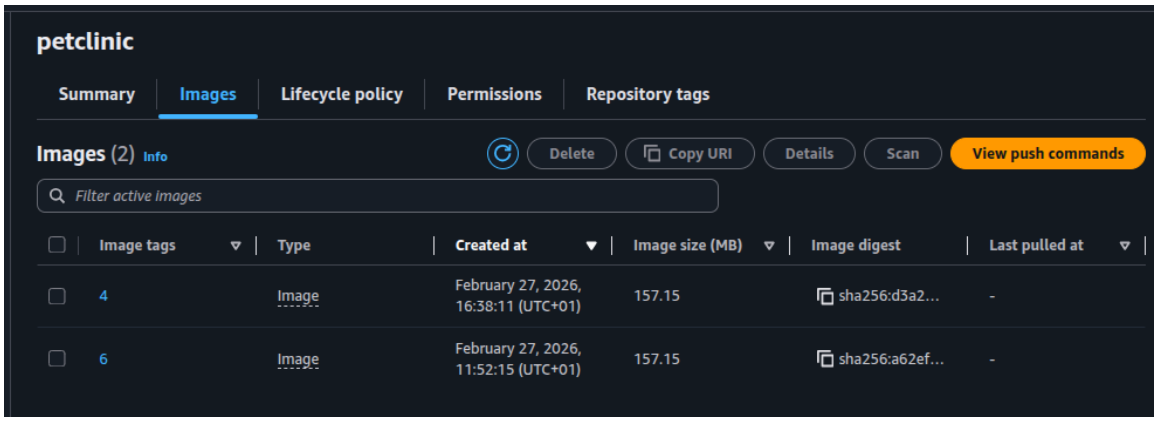

FIGURE 6.1 – AWS ECR — Repository `petclinic` avec deux images

Deux images ont été publiées automatiquement par les deux pipelines :

Tag	Date de création	Taille	Source
6	27 fév. 2026, 11 :52	157.15 MB	Jenkins
4	27 fév. 2026, 16 :38	157.15 MB	AWS CodeBuild

Les deux images ont la même taille (157.15 MB), ce qui confirme que les deux pipelines produisent un artefact identique à partir du même code source.

7. Comparaison Jenkins vs AWS CodePipeline

Critère	Jenkins	AWS CodePipeline
Setup time	∼2h (config manuelle)	∼20 min (console)
Maintenance	Élevée (self-hosted)	Nulle (managé)
Coût	Gratuit (self-hosted)	Pay per use
Scalabilité	Manuelle	Automatique
Intégration AWS	Plugin requis	Native
Flexibilité	Très élevée	Moyenne
Analyse qualité	SonarQube intégré	Non inclus nativement

7.1 Analyse

Les deux pipelines produisent le même résultat final : une image Docker testée et publiée dans AWS ECR. La différence n'est pas dans l'artefact, elle est dans tout ce qui entoure sa production.

Jenkins demande un investissement initial significatif : configuration des plugins, gestion du réseau Docker, mise en place des credentials, écriture du Jenkinsfile. Cet investissement est amorti sur la durée sur des projets complexes où la flexibilité est prioritaire. C'est le choix naturel pour des équipes avec un DevOps dédié, des environnements multi-cloud, ou un besoin d'intégration SonarQube native.

AWS CodePipeline inverse ce rapport : la configuration initiale est très rapide (connexion GitHub, déclaration du buildspec.yml, IAM Role) mais la personnalisation est plus limitée. L'absence de SonarQube natif est la limite la plus visible. En revanche, zéro infrastructure à maintenir et un déclenchement automatique fiable en font le choix idéal pour une équipe déjà sur AWS qui veut livrer vite sans charge opérationnelle.

La conclusion pratique est la suivante : **le vrai coût de Jenkins n'est pas financier, c'est humain**. Ce coût est invisible dans un tableau comparatif, mais très réel au quotidien dans une équipe.

8. Résultats & Conclusion

8.1 Résultats obtenus

— **57 tests unitaires** passant dans les deux pipelines.
— **SonarQube Quality Gate : Passed** — 0 bugs, 0 vulnérabilités.
— **Images Docker** publiées automatiquement dans AWS ECR à chaque build.
— **Auto-trigger** fonctionnel sur push GitHub dans AWS CodePipeline.

8.2 Difficultés rencontrées

Plusieurs problèmes concrets ont été rencontrés et résolus au cours du projet.

Permissions IAM pour le push ECR depuis Jenkins. Le premier push Docker vers ECR a échoué avec une erreur `no basic auth credentials`. La cause : le conteneur Jenkins n'avait pas accès aux credentials AWS. La solution a été de stocker les clés d'accès IAM comme credentials Jenkins et de les injecter dans le pipeline via la directive `withCredentials`. Une approche plus robuste en production serait d'utiliser un IAM Role attaché directement à l'instance EC2 hébergeant Jenkins.

Docker-in-Docker dans Jenkins. Le stage *Docker Build* a nécessité que le daemon Docker de la machine hôte soit accessible depuis le conteneur Jenkins. Cela a été résolu en montant le socket Docker (`/var/run/docker.sock`) dans le conteneur via le `docker-compose.yml`.

Syntaxe du buildspec.yml pour CodeBuild. La configuration initiale du `buildspec.yml` a provoqué plusieurs échecs en phase `PRE_BUILD` : les variables d'environnement `AWS_ACCOUNT_ID` et `AWS_DEFAULT_REGION` n'étaient pas déclarées dans les paramètres du projet CodeBuild. Le débogage s'est fait via les logs CloudWatch.

Premier lancement de SonarQube. Au démarrage, SonarQube impose un changement de mot de passe obligatoire via l'interface web avant d'accepter les connexions API. Cette étape manuelle, peu documentée, a bloqué le pipeline lors de la première exécution du stage SonarQube Analysis.

8.3 Conclusion

Ce projet démontre qu'il n'existe pas de meilleur pipeline CI/CD dans l'absolu : il existe le pipeline le mieux adapté à un contexte donné. Jenkins et AWS CodePipeline produisent exactement le même artefact final, mais les chemins pour y arriver sont fondamentalement différents en termes de complexité, de maintenance et de flexibilité.

Ce projet constitue une base solide et reproductible. La prochaine étape naturelle est d'ajouter un stage de déploiement vers ECS ou EKS, pour évaluer les deux approches sur l'ensemble du cycle de vie d'une application, de la compilation jusqu'à la mise en production.